

The qLDPC Challenge

A public, automatically verified leaderboard for quantum LDPC codes

Unitary Foundation

`github.com/unitaryfoundation/qldpc-challenge`

This talk

1. Why quantum error correction, in five minutes
2. Quantum LDPC codes: beyond the surface code
3. Distance: the hard part, and how claims can be checked anyway
4. The Challenge: an autoresearch loop with a trusted referee

Why error correction at all

- ▶ Physical qubits err at rates $\sim 10^{-3}$ – 10^{-2} per operation; useful algorithms need 10^{-10} or better.
- ▶ Fix: encode k **logical** qubits redundantly into n **physical** qubits, measure carefully chosen parities (*syndromes*), and undo errors faster than they accumulate.
- ▶ **Threshold theorem**: below a constant physical error rate, logical errors can be suppressed exponentially in the code distance d .
- ▶ The whole game is the **overhead**: how many physical qubits per logical qubit, at a given protection level?

Stabilizer codes and the CSS recipe

A CSS code is specified by two binary parity-check matrices:

$$H_X \in \mathbb{F}_2^{r_X \times n}, \quad H_Z \in \mathbb{F}_2^{r_Z \times n}, \quad \text{with } H_X H_Z^T = 0 \pmod{2}.$$

- ▶ Each row of H_X is an X -type parity check (a stabilizer); each row of H_Z a Z -type one. The orthogonality condition makes them all commute.
- ▶ Number of logical qubits:

$$k = n - \text{rank } H_X - \text{rank } H_Z.$$

- ▶ Everything about the code is *recomputable* from (H_X, H_Z) — a fact the Challenge leans on heavily.

Errors, split in two

Bit flips (X) are caught by Z -checks; phase flips (Z) by X -checks. A CSS code is two classical codes stitched together at the commutation condition.

The benchmark: the surface code

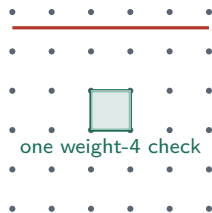
- ▶ Qubits on a 2D grid; every check touches only its **4 neighbours** $\Rightarrow$ easy to build and measure.
- ▶ Distance grows with the lattice: $d \sim \sqrt{n}$.
- ▶ But it encodes **one** logical qubit per patch:

$$k/n \rightarrow 0.$$

- ▶ A useful machine needs *thousands* of patches — millions of physical qubits.

Can we keep the low-weight checks but get a constant rate?

a logical operator: weight d



Quantum LDPC codes

LDPC = low-density parity checks: every check touches $O(1)$ qubits and every qubit sits in $O(1)$ checks — the property that makes syndrome measurement practical.

- ▶ Breakthrough (2020s): families with **constant rate and linear distance** exist — asymptotically “good” quantum codes.
- ▶ Practical frontier: small-to-medium block codes that beat the surface code’s overhead *today*, e.g. IBM’s bivariate-bicycle “gross code” $[[144, 12, 12]]$: 12 logical qubits at distance 12 from 144 data qubits, weight-6 checks.
- ▶ Headline figure of merit used throughout the Challenge:

$$\text{efficiency} = \frac{k d^2}{n} \quad (\text{surface code} \approx 1, \quad \text{gross code} = 12, \quad \text{board record} = 21.78).$$

Why kd^2/n ? A yardstick calibrated to two dimensions

Bravyi–Poulin–Terhal bound (2010)

Any code whose checks are geometrically local in a 2D layout obeys

$$k d^2 \leq c n \quad \Longleftrightarrow \quad \frac{k d^2}{n} \leq c,$$

with c a constant depending only on the check range — not on n .

- So for **2D-local codes the metric cannot grow**: the surface code ($k = 1$, $d \sim \sqrt{n}$) already *saturates the scaling* at $kd^2/n \approx 1$; better 2D-local codes only improve the constant (the Challenge's 2d-local tracks live here).
- Which is exactly why it makes a good yardstick: $kd^2/n \gg 1$ **quantifies how far beyond a strict 2D layout** a code performs. The board record of 21.78 is a statement about connectivity, not just cleverness.

Dimensionality raises the ceiling

BPT generalizes to checks local in D dimensions: $k d^{2/(D-1)} \leq O(n)$ and $d \leq O(n^{(D-1)/D})$
 $\Rightarrow kd^2/n \leq O(n^{2(D-2)/D})$.

Layout	BPT tradeoff	Max d	Ceiling on kd^2/n
2D local	$kd^2 \leq O(n)$	$O(\sqrt{n})$	$O(1)$
3D local	$kd \leq O(n)$	$O(n^{2/3})$	$O(n^{2/3})$
all-to-all	—	$\Theta(n)$	$\Theta(n^2)$

- ▶ The ceilings are **achievable**, not just bounds: BPT-saturating constructions exist in every D (Portnoy '23; Williamson–Baspın; Lin–Wills–Hsieh); “good” codes reach $k, d = \Theta(n)$ (Panteleev–Kalachev '21).
- ▶ So kd^2/n is a **constant-vs-growing diagnostic**: flat for planar hardware, polynomially higher with every dimension of long-range connectivity.
- ▶ On the leaderboard this needs **no new metric**: locality class is computed from the layout, kd^2/n compared only *within* a class — a 3d-local track would slot in unchanged.

One structure, three generalizations

Most board entries are **two-block** codes:

$$H_X = [A \mid B], \quad H_Z = [B^T \mid A^T], \quad AB = BA \Rightarrow H_X H_Z^T = 0.$$

- ▶ **Bivariate bicycle (BB)**: A, B are sparse polynomials in two commuting cyclic shifts x, y on a torus $\mathbb{Z}_\ell \times \mathbb{Z}_m$ — commutativity is free.
- ▶ **2BGA**: replace the torus by *any* finite group G ; A acts by left multiplication, B by right multiplication, which always commute. Non-abelian G (dihedral, metacyclic) reaches parameters BB cannot, including odd k .
- ▶ **Coset codes**: qubits on cosets G/H — the current record-efficiency family ($kd^2/n = 21.78$ at $[[180, 20, 14]]$).

Separate track: **2D-local planar** codes with open boundaries — qLDPC codes you could lay out on a chip bilayer, checks within radius ~ 3 on the grid.

Distance: what it is

A **logical operator** acts on the encoded qubits without triggering any check:

$$d_X = \min \left\{ \text{wt}(v) : H_Z v = 0, v \notin \text{rowspace}(H_X) \right\}, \quad d = \min(d_X, d_Z).$$

- ▶ In the kernel: commutes with every Z -check. Outside the rowspace: not itself a product of stabilizers.
- ▶ d is the weight of the *cheapest undetectable error* — it sets the protection level, and it enters the figure of merit *squared*.
- ▶ **Computing d exactly is NP-hard.** Every distance number you read in a paper was produced by a bounded search — and bounded searches miss things.

Measuring distance in practice

Upper bounds: find a logical

- ▶ *Randomized information-set search*: RREF the kernel basis under random column orders; light rows (and their pairwise sums) are candidate logicals.
- ▶ *Decoder residuals*: inject errors, decode with BP+OSD; a decoding failure that commutes with all checks is a logical operator.
- ▶ Either way the output is an explicit **witness**: a vector anyone can check in milliseconds. It proves $d \leq \text{wt}(v)$ — trustlessly.

Exact values: prove nothing lighter exists

- ▶ Integer program per logical class: minimize $\text{wt}(v)$ subject to $H_Z v = 0 \pmod{2}$ and $\ell \cdot v = 1 \pmod{2}$.
- ▶ Feasible for small / low-rate codes; hopeless for large high-rate ones.

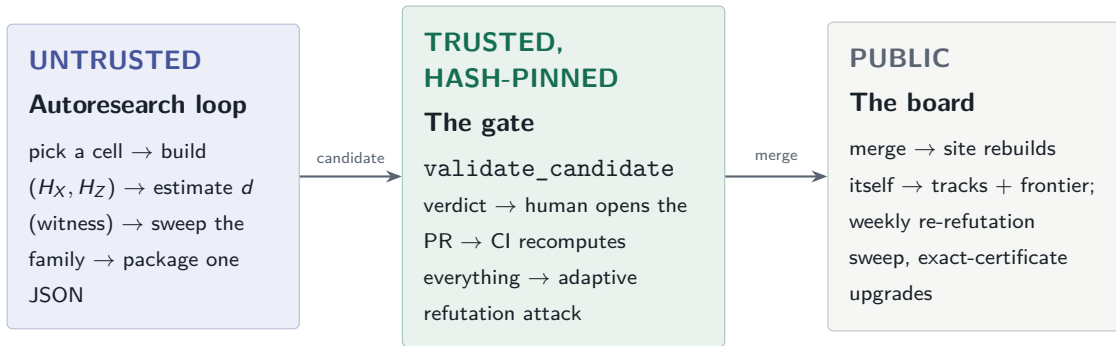
The trap

A *high* estimate from a *short* search looks like a great code. It usually means the search hasn't found the light logical yet. Distance claims need adversarial checking.

What the Challenge is

- ▶ A **public leaderboard** of qLDPC codes: one JSON file per code, in a git repository, with a static site on top. 41 codes today
- ▶ Every claim is **recomputed from the parity checks** — n , k , CSS commutation, check weights, distance witnesses, 2D-locality radius. Nothing is trusted from the submitter.
- ▶ Codes compete on **Pareto frontiers** inside track cells (check-weight class $\times$ locality class), never on a single score.
- ▶ Designed so that **LLM research agents** can do the searching — with a referee they cannot talk their way past.

Three zones, one direction of trust



The one rule (from the agent manual, `AUTORESEARCH.md`)

No code is a find until `validate_candidate` returns `passed: true`. You may write and run any code you like — but you do not get to decide whether a code is good.

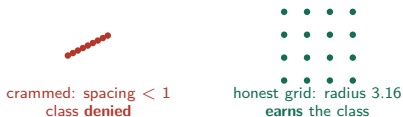
The CI gauntlet, per submission

1. **Scope:** a PR adding a code may not touch workflows, the schema, or `verify/` — a submission cannot change its own judge. One new code per PR.
2. **Integrity pin:** SHA-256 of the whole trusted stack against a committed manifest; any drift fails the build.
3. **Self-tests:** the verifier must still *reject* known-bad submissions and known over-claims.
4. **Recomputation:** n , k , CSS, weights, witnesses, locality — recomputed by the *base branch's* verifier.
5. **Adaptive refutation:** two independent searches attack the distance claim. Budget scales with size $(2500 + 40n)$ trials; a claim that advances a frontier pays $3 \text{ seeds} \times (25000 + 120n)$ trials before it earns the record. One lighter logical found $\Rightarrow$ PR fails, witness printed.
6. **Authorship:** the PR author must be a listed author of the code.

Geometry is verified too: you cannot lie about locality

The 2D-local tracks matter (they carry the $kd^2/n \leq c$ ceiling), so class membership is **computed, never declared**: a submission carries its layout — coordinates plus a layer count — as a *checkable certificate*, exactly like the distance witness.

- ▶ a coordinate for **every** qubit;
- ▶ **no cramming**: at most layers qubits per site, distinct sites ≥ 1 apart — a small radius cannot be faked by collapsing or rescaling points;
- ▶ the interaction radius is **measured** (largest check diameter) and must sit under the class cap;
- ▶ a dishonest or missing layout is not rejected — the code simply competes as unrestricted, the harder cell.



Field test: retro-tagging seven planar codes, the first (wrong) embedding measured radius 10.2, growing with size — refused. The correct one measured 3.16, size-independent — earned.

Distance claims are tiered by how they were earned

Tier	Meaning	Earned by
$d \leq v$	upper bound	witness checked trustlessly, milliseconds
$d = v$	exact	server MILP certificate committed to certs/
refuted	claim killed	any search finding a lighter logical, witness attached

- ▶ 27 of 41 board codes hold an exact certificate.
- ▶ The other defense is time: a **weekly random-seed sweep** re-attacks every entry on the board, forever.

Case study: the pipeline correcting an over-claim

[[294, 8, 19]], a **2BGA code on a metacyclic group** (June 2026)

1. An LLM agent sweeping metacyclic 2BGA codes surfaced a candidate claiming $d = 20$ (with a Z -side claim of 25). It *passed* the standard 8k-trial gate.
2. Deep refutation — 60k trials across independent seeds — found a **weight-19 Z -logical**: the claim was an artifact of an under-converged upper bound.
3. Resubmitted with the honest $d = 19$ *and the found witness attached*; the X -side value corroborated by every deep seed.
4. Merged as a non-dominated weight-8 entry — and the incident sized the adaptive deep gate that now guards every frontier claim.

The system's job is not to prevent optimistic claims — it is to make sure they don't survive.

What it is for

- ▶ A **shared, machine-checked record** of which qLDPC codes are best under which constraints — parameters today live scattered across papers, hard to reproduce or compare.
- ▶ A **benchmark for AI research agents**: the starter kit (`research/`) is tool-agnostic, pure numpy, and sits on the verifier's own linear algebra; `AUTORESEARCH.md` is an operating manual any model can be pointed at.
- ▶ A template for **verification-first open science**: untrusted exploration, a pinned referee, and claims that decay gracefully (refuted with evidence) instead of persisting as folklore.

Beat the frontier: github.com/unitaryfoundation/qldpc-challenge

Thanks!

Live board, starter kit, and the full verification stack:

`github.com/unitaryfoundation/qldpc-challenge`

`unitaryfoundation.github.io/qldpc-challenge`